has a category name starting with `Microsoft .AspNetCore` will only generate logs at the `Warning` level or above.

You can override the rules specified in the `LogLevel` section by specifying new rules that apply to individual providers. In the next listing, add a section for the `Console` provider that results in logs being generated at `Debug` level and above, except for those in the Microsoft category, where you are only interested in seeing errors or critical events being recorded.

Listing 12.13 Adding a provider-specific filter rule

```
"Logging": {
  "LogLevel": {
    "Default": "Information",
    "Microsoft.AspNetCore": "Warning"
  },
  "Console": {                    ❶
    "LogLevel": {
      "Default": "Debug",         ❷
      "Microsoft": "Error"        ❸
    }
  }
}
```

❶ New rules that only apply to the Console logger

❷ The default minimum logging level is set to Debug.

❸ The minimum logging level for categories starting with Microsoft is Error.

The `Console` section applies only to the console logger. Other loggers—the debug logger and the event source logger, for example—do not have specific rules, so their behavior is governed by the settings in the `LogLevel` section.

We have looked at working with the default logging implementations, but they are limited in terms of their application. When your website is live, you need to know when a show-stopping error has occurred. You also need